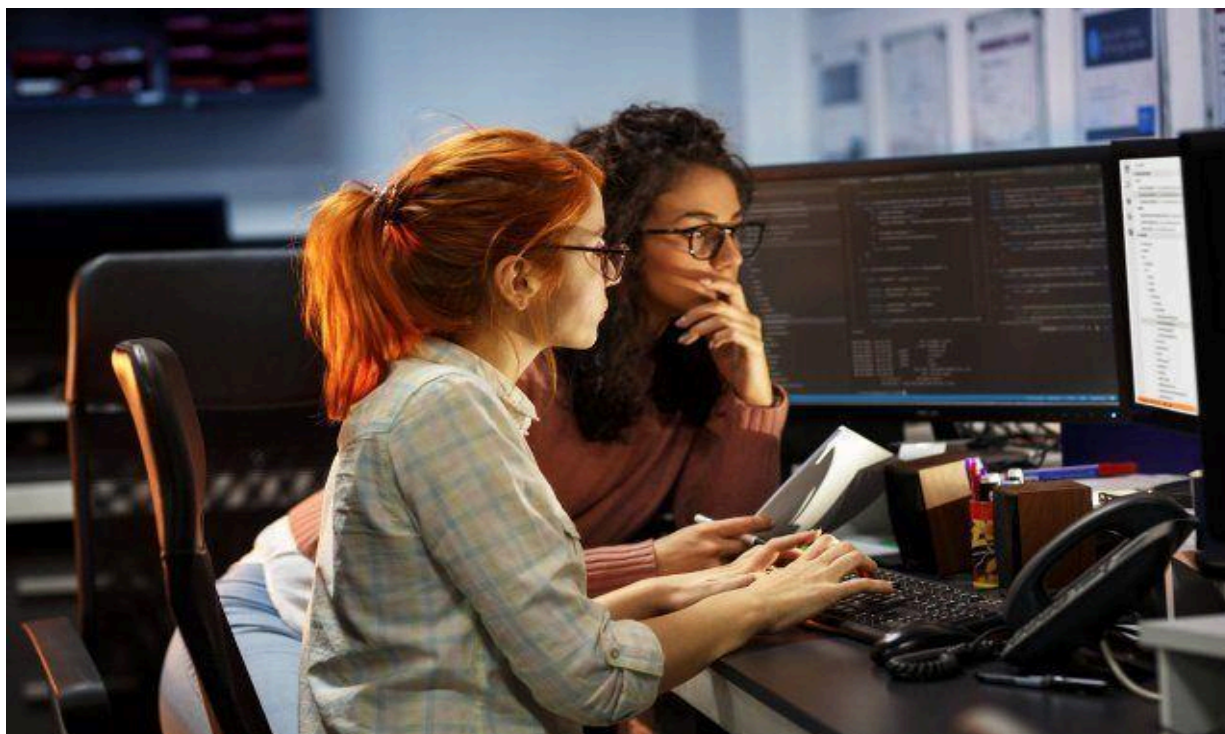




Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Asignatura:
Programación Orientada a Objetos

Unidad #: 3

Tema: Programacion Orientada a Objetos en Desarrollo Moderno.

SEMANA 9

Autor:

Introducción:

Bienvenidos al material de lectura #1 de la Unidad #3 de la materia Programación Orientada a Objetos. En este documento exploramos los conceptos de Spring, spring boot y REST, uso de postman para probar tu primer API y Lombok para ahorrarte código. Este material ha sido diseñado para brindarte una base sólida en el ámbito del desarrollo de software, preparándote para afrontar los retos y oportunidades que surgirán en tu trayectoria profesional. A medida que avances en el contenido, te animamos a involucrarte activamente en los ejemplos, ventajas, limitaciones y ejercicios propuestos, ya que contribuirán significativamente a tu proceso de aprendizaje.

Semana 9

3. Introducción a Spring Boot y REST

Ahora que sabes cómo construir objetos y clases en Java, es hora de hacer que tus aplicaciones se comuniquen con el mundo exterior. En esta unidad, aprenderás a crear APIs REST utilizando Spring Boot, una de las herramientas más populares y potentes en el ecosistema de Java.

3.1 ¿Qué es Spring Framework y qué aporta Spring Boot?

Para entender Spring Boot, primero debemos hablar de su padre: **Spring Framework**.

Spring Framework es una caja de herramientas gigante y poderosa para construir aplicaciones empresariales en Java. Proporciona soluciones para casi todo: seguridad, acceso a bases de datos, transacciones y, lo más importante, un principio llamado **Inyección de Dependencias (DI)**, que nos ayuda a escribir código más limpio y desacoplado.

- **Analogía:** Piensa en **Spring Framework** como una **enorme tienda de materiales de construcción**. Te da acceso a vigas de la más alta calidad, el mejor cableado eléctrico y tuberías de primera. Sin embargo, tú, como arquitecto, tienes que conectar todo manualmente. Es increíblemente potente, pero la configuración inicial puede ser compleja y llevar mucho tiempo.

Spring Boot es una evolución de Spring Framework que busca eliminar toda esa configuración compleja.

- **Analogía:** Si Spring Framework es la tienda de materiales, **Spring Boot** es un **kit de casa prefabricada de alta calidad**. Ya viene con las paredes ensambladas, los planos del sistema eléctrico definidos y un manual simple. Simplemente unes las piezas y tienes una casa funcional en una fracción del tiempo. Aún puedes personalizarla y cambiar los materiales si quieres, pero todo el trabajo pesado ya está hecho.

Spring Boot logra esto gracias a tres ideas clave:

1. **Autoconfiguración:** Spring Boot mira las librerías que has añadido a tu proyecto y automáticamente configura la aplicación por ti. ¿Añades la librería para bases de datos? Spring Boot configura la conexión. ¿Añades la librería web? Spring Boot configura el servidor.

2. **Servidor Web Embebido:** No necesitas instalar y configurar un servidor web como Tomcat por separado. Lo incluye dentro de tu aplicación. Simplemente ejecutas tu programa y ya tienes un servidor funcionando.
3. **"Starters":** Simplifican la gestión de dependencias. En lugar de añadir 20 librerías para una aplicación web, solo incluyes una dependencia llamada `spring-boot-starter-web`, y esta se encarga de traer todo lo demás que necesitas.

3.2 ¿Qué es una API REST? Principios y recursos.

Una **API** (Interfaz de Programación de Aplicaciones) es un contrato que permite que dos piezas de software se comuniquen. **REST** (Representational State Transfer) es un estilo arquitectónico, un conjunto de reglas y principios para diseñar APIs que sean fáciles de entender y usar.

- **Analogía:** Piensa en una API REST como el **menú de un restaurante**.
- **Recursos (Los Platos del Menú):** Un recurso es cualquier "cosa" con la que un cliente quiera interactuar. Son los **sustantivos** de tu API. En un menú, los platos son los recursos. En una API, serían `/productos`, `/usuarios`, `/pedidos`.
- **Verbos HTTP (Las Acciones):** Son las acciones que puedes realizar sobre esos recursos.
 - **GET: Consultar** un recurso. (Ver la descripción de un plato).
 - **POST: Crear** un nuevo recurso. (Hacer un nuevo pedido).
 - **PUT: Actualizar** un recurso existente. (Cambiar tu pedido).
 - **DELETE: Eliminar** un recurso. (Cancelar tu pedido).
- **Representaciones:** Cuando pides un plato, te lo traen en una forma específica (en un plato, con cubiertos). En una API REST, la representación más común de los datos es **JSON** (JavaScript Object Notation), un formato de texto ligero y fácil de leer tanto para humanos como para máquinas.

3.3 Estructura de un proyecto Spring Boot.

Un proyecto Spring Boot estándar, usando una herramienta de construcción como Maven, tiene la siguiente estructura:

`mi-proyecto/`



```
└─ pom.xml // El "plano" del proyecto, define las dependencias (starters).
```

```
└─ src/
    └─ main/
        └─ java/
            └─ com/
                └─ ejemplo/
                    └─ miproyecto/
                        └─ MiProyectoApplication.java // El punto de entrada principal.
                    └─ resources/
                        └─ application.properties // Archivo para configurar tu aplicación.
                └─ test/
                    └─ java/ // Aquí van tus pruebas unitarias.
```

- **pom.xml**: Es el corazón de un proyecto Maven. Le dice a Java qué "starters" y otras librerías necesita tu proyecto.
- **MiProyectoApplication.java**: La clase con el método **main** que arranca toda la aplicación Spring Boot.
- **application.properties**: Un archivo simple de clave-valor donde configuras cosas como el puerto del servidor, la conexión a la base de datos, etc.

3.4 Anotaciones básicas de Spring Boot

Las anotaciones son "etiquetas" que ponemos en nuestro código para decirle a Spring Boot cómo debe comportarse.

Anotaciones Principales

- **@SpringBootApplication**: Se coloca en la clase con el método `main` y es el punto de entrada que activa toda la magia de Spring Boot (autoconfiguración, escaneo de componentes, etc.).

Anotaciones para Controladores (Capa Web)

- **@RestController**: Marca una clase como un controlador web que devolverá datos (como JSON) directamente en el cuerpo de la respuesta.
- **@RequestMapping("/ruta-base")**: Define una URL base para todos los métodos dentro de un controlador.
- **@GetMapping("/sub-ruta")**: Atajo para **@RequestMapping(method = RequestMethod.GET)**. Mapea peticiones HTTP GET.
- **@PostMapping**: Atajo para peticiones POST (crear recursos).
- **@PutMapping**: Atajo para peticiones PUT (actualizar recursos).
- **@DeleteMapping**: Atajo para peticiones DELETE (eliminar recursos).

Anotaciones para Leer Datos de la Petición

- **@PathVariable**: Extrae un valor de la propia URL. Por ejemplo, en `/productos/{id}`, **@PathVariable Long id** tomaría el valor del `id`.
- **@RequestParam**: Extrae un parámetro de la query string. Por ejemplo, en `/productos?nombre=Laptop`, **@RequestParam String nombre** tomaría el valor "Laptop".
- **@RequestBody**: Le dice a Spring que convierta el cuerpo de la petición (generalmente un JSON) en un objeto Java. Es esencial para **POST** y **PUT**.

Anotaciones para la Estructura y Dependencias

- **@Service**: Marca una clase como un servicio de negocio. Es donde suele ir la lógica principal de la aplicación.
- **@Repository**: Marca una clase que se encarga del acceso a datos (hablar con la base de datos).
- **@Autowired**: Le dice a Spring que "inyecte" automáticamente una dependencia. Por ejemplo, para usar una clase de servicio dentro de un controlador.

@Autowired es fundamental para conectar las diferentes capas de tu aplicación sin tener que crear las instancias tú mismo. **@Autowired** es opcional si solo hay una forma de construir el objeto (un constructor). Este proceso se llama **Inyección de Dependencias**.

Usando @Autowired

Aquí tienes un ejemplo práctico y claro que separa la lógica de negocio (en un @Service) de la lógica web (en un @RestController).

Imagina que queremos crear un endpoint que devuelva un saludo personalizado. En lugar de poner la lógica para crear el saludo directamente en el controlador, la encapsularemos en una clase de servicio. El controlador simplemente le pedirá al servicio que haga el trabajo.

Antes de iniciar, repasemos:

- **Capa Model**

Responsabilidad: Son simples clases Java (POJOs) que representan los datos de tu aplicación.

- **Capa Controller**

Responsabilidad: Mostrar información al usuario y capturar sus acciones. Es la única capa con la que el usuario interactúa directamente.

Qué contiene: Controladores REST (@RestController), vistas HTML/CSS/JavaScript, componentes de frameworks de frontend (React, Angular).

- **Capa Service**

Responsabilidad: Orquestrar el flujo de la aplicación y ejecutar las reglas de negocio. Es el cerebro del sistema.

Qué contiene: Clases de Servicio (@Service) que validan datos, realizan cálculos y coordinan las acciones entre la capa de presentación y la de datos

- **Capa Repository**

Responsabilidad: Encargarse de toda la comunicación con la base de datos. Su única tarea es abstraer las operaciones de almacenamiento (CRUD: Crear, Leer, Actualizar, Borrar).

Qué contiene: Repositorios (@Repository), DAOs (Data Access Objects), y el código que traduce objetos Java a tablas de base de datos y viceversa.

Ahora sí, comencemos.

Paso 1: Crear la Capa de Servicio (@Service)

Esta clase se encargará de la "lógica de negocio", que en este caso es simplemente generar un mensaje.

1. Crea una nueva clase llamada `GreetingService.java`.
2. Añade la anotación `@Service`. Esto le dice a Spring: "Gestiona esta clase, crea una instancia de ella y tenla lista para cuando alguien la necesite".

```
package com.example.demo;
```

```
import org.springframework.stereotype.Service;
```

```
@Service // 1. Marcamos esta clase como un componente de  
servicio gestionado por Spring.
```

```
public class GreetingService {  
  
    /**  
     * Lógica de negocio para crear un saludo.  
     * @param name El nombre a saludar.  
     * @return Un mensaje de saludo personalizado.  
     */  
  
    public String getGreetingMessage(String name) {  
        if (name == null || name.trim().isEmpty()) {  
            return "¡Hola, mundo!";  
        }  
  
        return "¡Hola, " + name + "!";  
    }  
}
```

```
}  
  
}
```

Paso 2: Crear la Capa del Controlador (@RestController)

Ahora, el controlador usará el `GreetingService` para obtener el mensaje y devolverlo al usuario. Aquí es donde entra en juego `@Autowired`.

1. Crea o modifica tu controlador, por ejemplo, `GreetingController.java`.
2. Declara una variable para `GreetingService`.
3. Usa `@Autowired` para que Spring "inyecte" automáticamente la instancia del servicio que ya creó.

```
package com.example.demo;
```

```
import  
org.springframework.beans.factory.annotation.Autowired;  
  
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.RequestParam;  
  
import  
org.springframework.web.bind.annotation.RestController;
```

```
@RestController
```

```
public class GreetingController {
```

```
    // 2. Declaramos la dependencia que necesitamos.
```

```
    private final GreetingService greetingService;
```



```
// 3. ¡Aquí ocurre la magia!

// Le decimos a Spring que, para crear este controlador,
// necesita "inyectar" una instancia de GreetingService.

@Autowired

public GreetingController(GreetingService
greetingService) {

    this.greetingService = greetingService;

}

// 4. El endpoint ahora usa el servicio para hacer el
trabajo.

@GetMapping("/saludo")

public String saludar(@RequestParam(value = "nombre",
required = false) String nombre) {

    // El controlador ya no sabe CÓMO se crea el saludo,
    solo le pide al servicio que lo haga.

    return greetingService.getGreetingMessage(nombre);

}

}
```

¿Qué está Pasando Detrás de Escena?

1. Al iniciar la aplicación, Spring escanea tu código y encuentra la clase `GreetingService` marcada con `@Service`. Automáticamente, crea una única instancia de esta clase y la guarda en su "contenedor de beans".

2. Luego, encuentra la clase `GreetingController`. Ve que su constructor está marcado con `@Autowired` y que necesita un `GreetingService` para poder ser creada.
3. Spring busca en su contenedor la instancia de `GreetingService` que ya había creado y se la pasa al constructor del `GreetingController`.

Sin `@Autowired`, tendrías que hacer esto manualmente dentro del controlador:
`private GreetingService greetingService = new GreetingService();` Esto crea un **acoplamiento fuerte** y va en contra de los principios de Spring. Con `@Autowired`, tu controlador no es responsable de crear sus dependencias, solo de declararlas.

Analogía del Chef 🍳

Piensa en el `GreetingController` como un **Chef**. El `GreetingService` es un **Proveedor de Ingredientes**.

- **Sin Inyección de Dependencias:** El Chef tendría que ir personalmente al mercado a buscar cada ingrediente (`new GreetingService()`). Es lento y si el mercado cambia, el Chef tiene problemas.
- **Con Inyección de Dependencias (`@Autowired`):** El Chef simplemente le dice a su asistente (Spring): "Necesito ingredientes". El asistente sabe exactamente qué proveedor (`GreetingService`) llamar y le trae los ingredientes listos para usar. El Chef se enfoca solo en cocinar.

3.6 Creación de un servicio REST (Ejemplo Completo)

Vamos a crear un controlador más realista para gestionar una lista de productos.

1. Primero, crea una clase simple para el producto (`Product.java`):

```
// Ejemplo de esqueleto para Product.java
```

```
package com.miempresa.productosapi;
```

```
public class Product {
```

```
    private Long id;
```

```
private String name;
private double price;

public Product() {
    // Constructor vacío es útil para la deserialización
    JSON
}

public Product(Long id, String name, double price) {
    this.id = id;
    this.name = name;
    this.price = price;
}

// --- Getters ---
public Long getId() { return id; }
public String getName() { return name; }
public double getPrice() { return price; }

// --- Setters ---
public void setId(Long id) { this.id = id; }
public void setName(String name) { this.name = name; }
public void setPrice(double price) { this.price = price; }
}
```

2. Ahora, crea el controlador (**ProductController.java**):

```
// Ejemplo de esqueleto para ProductController.java

package com.miempresa.productosapi;

import org.springframework.web.bind.annotation.*;
import java.util.ArrayList;
import java.util.List;

@RestController
@RequestMapping("/api/products")
public class ProductController {

    // Simulación de una base de datos en memoria
    private List<Product> products = new ArrayList<>();

    public ProductController() {
        products.add(new Product(1L, "Laptop XYZ", 1200.00));
        products.add(new Product(2L, "Mouse Gamer", 75.50));
        products.add(new Product(3L, "Teclado Mecánico",
150.00));
    }

    // Endpoint 1: Obtener todos los productos
    @GetMapping
    public List<Product> getAllProducts() {
```

```
        return products;  
    }  
}
```

Para probarlo:

1. Ejecuta tu aplicación Spring Boot.
2. Usa un navegador para probar los **GET**:
`http://localhost:8080/api/products`.
3. Para probar el **POST**, necesitarás una herramienta como **Postman** o **Insomnia**, ya que los navegadores no pueden hacer peticiones POST fácilmente.

Postman: Probando tu API como un Profesional

¿Qué es Postman?

Postman es una aplicación gráfica que te permite crear y enviar cualquier tipo de petición HTTP. Es la herramienta estándar para probar, documentar y desarrollar APIs.

¿Por Qué lo Usamos? (El Problema que Resuelve)

Un navegador web es muy limitado para probar una API.

- Solo puede hacer peticiones **GET** fácilmente.
- No puedes enviar un cuerpo (body) en formato JSON para probar un **POST** o **PUT**.
- No puedes configurar cabeceras (headers) personalizadas.

Postman te da control total sobre la petición que quieres enviar a tu API.

Cómo Probar Nuestra API de Productos con Postman:

1. **Descarga e instala** Postman desde su [sitio web oficial](#).
2. **Inicia tu aplicación Spring Boot** para que el servidor esté corriendo en `localhost:8080`.
3. **Prueba el endpoint GET /api/products:**
 - Abre Postman y crea una nueva petición.

- Deja el método como **GET**.
 - En la barra de URL, escribe:
`http://localhost:8080/api/products`
 - Haz clic en el botón **"Send"**.
4. **Resultado:** En la parte inferior de la ventana, en la pestaña "Body", verás la respuesta de tu API: un array de objetos **Product** en formato JSON y un estado **200 OK**.
[Imagen de la interfaz de Postman mostrando una petición GET a /api/products y la respuesta JSON]
5. **(Ejercicio Avanzado) Cómo probarías un POST:** Si hubieras creado un endpoint `@PostMapping`, lo probarías así:
- Cambia el método a **POST**.
 - Ve a la pestaña **"Body"** debajo de la URL.
 - Selecciona la opción **raw** y en el menú desplegable elige **JSON**.
 - En el área de texto, escribe el JSON del nuevo producto que quieres crear:
JSON

```
{  
  
  "id": 4,  
  
  "name": "Monitor 4K",  
  
  "price": 450.00  
  
}
```

- Haz clic en **"Send"**.
6. Postman enviaría esta petición a tu API, y podrías verificar si el nuevo producto se crea correctamente.

Lombok: Simplificando tu Código Java

¿Qué es Lombok?

Lombok es una librería de Java que se conecta a tu editor y a tus herramientas de construcción para **generar automáticamente código repetitivo** (boilerplate) por ti. En lugar de escribir métodos getters, setters, constructores, `toString()`, etc., simplemente usas una anotación.

¿Por Qué lo Usamos? (El Problema que Resuelve)

Las clases de modelo (como nuestra clase `Product`) pueden volverse muy largas y difíciles de leer. La mayor parte del código son métodos que siempre son iguales.

Antes de Lombok, nuestra clase `Product` se veía así:

```
public class Product {  
  
    private Long id;  
  
    private String name;  
  
    private double price;  
  
    public Product() {}  
  
    public Product(Long id, String name, double price) {  
        this.id = id;  
        this.name = name;  
        this.price = price;  
    }  
  
    public Long getId() { return id; }
```



```
public void setId(Long id) { this.id = id; }

public String getName() { return name; }

public void setName(String name) { this.name = name; }

public double getPrice() { return price; }

public void setPrice(double price) { this.price = price;
}
```

```
// ... y faltarían los métodos equals(), hashCode() y
toString()
```

```
}
```

¡Más de 20 líneas de código para solo 3 atributos!

Con Lombok, la misma clase se ve así:

Java

```
import lombok.AllArgsConstructor;
```

```
import lombok.Data;
```

```
import lombok.NoArgsConstructor;
```

```
@Data // Genera todos los getters, setters, toString, equals
y hashCode
```

```
@AllArgsConstructor // Genera el constructor con todos los
argumentos
```

```
@NoArgsConstructor // Genera el constructor vacío
```

```
public class Product {
```

```
    private Long id;
```

```
private String name;  
  
private double price;  
  
}
```

El resultado final es el mismo, pero el código es **mucho más limpio, legible y fácil de mantener**. Te concentras en los atributos, que es lo que realmente define a tu modelo.

